

BuildingSage: A safe and secure AI copilot for smart buildings

Anonymous Author(s)

ABSTRACT

The dawn of Generative AI is promising to deliver various benefits across all walks of life, including the built environment sector. While there are many opportunities, sharing proprietary and sensitive data with large generative AI models is seen as a risk when adopting them at the enterprise level. This paper introduces a privacy-preserving AI copilot architecture designed to facilitate the adoption of smart building technologies and tools while mitigating concerns related to data privacy. We demonstrate how the proposed AI copilot can effectively assist users in interacting with smart building systems, ensuring enhanced privacy and fostering broader adoption.

CCS CONCEPTS

• Information systems → Information integration; • Computing methodologies → Artificial intelligence.

KEYWORDS

LLM, Generative AI, smart buildings, semantic model construction, query

ACM Reference Format:

Anonymous Author(s). 2024. BuildingSage: A safe and secure AI copilot for smart buildings. In *Proceedings of November 07–08, 2024 (BuildSys '24)*. ACM, New York, NY, USA, 9 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Access to data and metadata in buildings in a standardised way is one of the main challenges limiting the implementation of data driven energy efficiency and cost saving initiatives in buildings. Building data interoperability challenges can be addressed by platforms that model real-world systems in standardised domain schema. The literature has examined metadata representations such as Project Haystack [11], SAREF [5], RealEstateCore [9] and Brick [3] with respect to their ability to capture salient metadata of building systems. Standard semantic metadata models that use these schemas provide a means for analytics providers to discover and extract relevant time-series data and metadata from a building to deploy their solutions.

While these methods have been demonstrated in various literature [7], [14], [18], [15], their scalability is hampered by lack of industry readiness to adopt these tools. Existing built environment industry eco-system has no role for a semantic model developer, a semantic model based platform operator or an analytics provider

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

BuildSys '24, Hangzhou, China,

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
<https://doi.org/XXXXXXX.XXXXXXX>

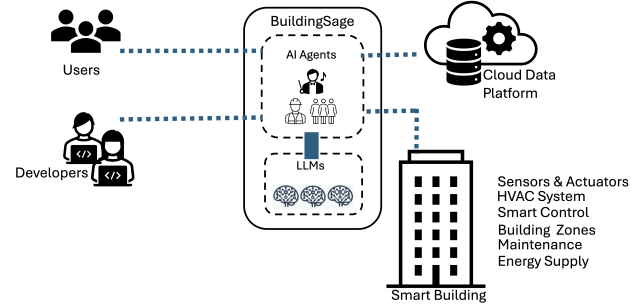


Figure 1: LLM-assisted smart building data interactions.

who can interact with these models. Existing industry practitioners do not possess the relevant skills to embrace these new methods limiting their uptake. The industry is in dire need of tools and processes that can help existing practitioners and users interact with smart building systems without coding expertise and knowledge of specialized schemas and tools such as Brick and SPARQL.

Recently, Large Language Models (LLMs) and Large Multimodal Models (LMMs) (e.g., GPT-4, GPT-4V, GPT-3, Llama, Mixtral 8x7B, Gemini, etc.) have been applied in various application domains (e.g., healthcare [4], finance [27], chatbots and virtual assistants [13], code generation and debugging [20], education [12], etc.), beyond Natural Language Processing (NLP) tasks, due to their advanced capabilities in information retrieval, reasoning, understanding and generating content across different modalities, and code generation. These foundation models have a large number of parameters, and they are pre-trained using vast amounts of data from diverse sources, contributing to their capacity to generate coherent and contextually relevant outputs across multiple modalities. The transformer architecture and attention mechanisms [25] are key components of large models, improving the efficiency of model training and the ability to handle long-range dependencies in the data. Using fine-tuning with specific datasets and prompt engineering, these large models can be further customized for domain-specific tasks.

There has been a growing interest in applying large generative models to smart building sector (e.g., data mining [30], chatbots for energy systems [8], efficiency and decarbonization [31], HVAC control [1], [22], and load forecasting [29]). However, integrating large generative models into this domain poses several challenges, including data handling related to sensitive data and privacy, maintaining consistency, mitigating hallucinations, and the inherent limitations of their general capabilities. Smart building systems use proprietary and sensitive data that can range from building system details, energy usage patterns, occupant behaviour in specific zones to name a few. Ensuring these information remain protected is crucial, especially when cloud-based large models are used to process and analyze the data. The large generative models need to provide consistent and reliable outputs to be effective in smart

building energy systems, as inconsistencies can lead to incorrect decisions and actions, which may negatively impact the outcomes. Furthermore, these models can *hallucinate*, generating plausible but incorrect responses, which may lead to erroneous recommendations and potential safety risks. The general capabilities of large generative models may not fully address the specific needs of smart building systems, requiring customization to handle the unique tasks and technical requirements. Finally, customization and integration require domain expertise in smart building energy and generative AI domains.

In this paper, we propose BuildingSage, an AI copilot that can assist developers and users in their interactions with smart building systems integrating various data sources and capabilities. BuildingSage protects the privacy of proprietary and sensitive data by sharing only the metadata (e.g., data format) rather than the raw data with the remote LLM model. To perform tasks, the co-pilot generates code, which is executed on the data locally. Furthermore, BuildingSage is based on a multi-agent framework, which improves the security, reliability, and accuracy of the system. Figure 1 illustrates the interactions among entities. The co-pilot enables users and developers to interact with the building systems using natural language, reducing the barriers to domain expertise and coding proficiency.

2 APPLICATION OF GENERATIVE AI IN SMART BUILDING ENERGY SYSTEMS

In this section, we explore the opportunities that Generative AI offers for both semantic model developers and semantic model users of smart building systems.

2.1 Opportunities for Developers

System Integrators (SI) traditionally install smart in a building system including setting up of building management systems. They possess operational knowledge of OT (Operational Technology) and Information Technology (IT) systems in buildings. They do not possess any knowledge of metadata schema and semantic models that link contextual information of buildings and time series data. Moreover these activities make the model construction a time and labour intensive exercise thus limiting its uptake. Generative AI can support them with the task of model creation in various ways.

- Extracting semantic information from building metadata sources such as drawings, floor plans to assist with creation of semantic models.
- Verification of semantic models once constructed.
- Updating and repairing semantic models.
- Ensuring data quality in operational systems.
- Data discovery in scenarios with a large number of data sources.
- Creating new datasets through data synthesis: In [2], authors proposed to synthesize daily household energy consumption patterns utilizing private LLMs to extract realistic datasets. In [21], an LLM-based SQL code generation approach was utilized for data transformation from source datasets into the target datasets for building energy data.
- Model-based continuous commissioning for energy benchmarking, FDD, etc.

2.2 Opportunities for Users

Generative AI empowers domain users and users with limited software background to develop and interact with smart building systems using natural language, eliminating the need to learn domain specific schema and or write scripts for setting up analytics:

- Analyzing and interpreting sensor data, in combination with traditional time series data analytics. In [30], Zhang et al. explored the potential of GPT-4 in data mining scenarios of building energy management for energy load prediction, fault diagnosis, and anomaly detection. In [29], a novel energy load forecasting approach that leverages language models was proposed. The proposed approach converts energy consumption data to descriptive sentences to fine-tune language models and generates energy load consumption forecasts using an autoregressive method.
- Interactive/non-expert fault diagnostics and detection. The user can interactively query the generative AI system that is built on top of smart energy systems to determine normal ranges and potential faults from sensor readings in a “free-style” fashion, so that the user experience is not bounded by prescribed fault detection rules or algorithms [23]. Such diagnosis can be set as scheduled tasks to be performed routinely.
- Setting up routines/preferences/rules for setpoints using natural language. Using natural language, with or without voice recognition, the setpoints can be configured with arbitrary routines/preferences/rules according to attributes and features in the spatio-temporal or numeric space. This would allow users to explicitly request configuring the smart building energy system to suit their day-to-day needs in a straightforward way [28].
- Improving transparency through interactive queries/feedback from the end user. Generative AI will help users by improving the transparency behind their energy expenditures. By interacting with their historical energy data through a conversational interface with generative AI, they will better understand their own patterns in energy usage [10]. With such reviews and reflections, users will also be able to identify potential opportunities for optimization.
- Automating energy expenditure optimization. Generative AI can be used in the optimization of energy expenditure in a few different ways. It can play the role of data analyst for the energy consumption to identify patterns, and it can offer suggestions to optimise and plan the general expenditure accordingly as a planner [24], using routines for example.

3 BUILDINGSAGE AI COPILOT

3.1 System Architecture

This section presents the AI copilot system designed to automate and enhance workflows in smart building energy systems. A detailed examination of the system’s architecture and components is provided. As depicted in Figure 2, the architecture outlines the interaction between the copilot’s components within a host machine, as well as their integration with external and local resources.

BuildingSage: A safe and secure AI copilot for smart buildings

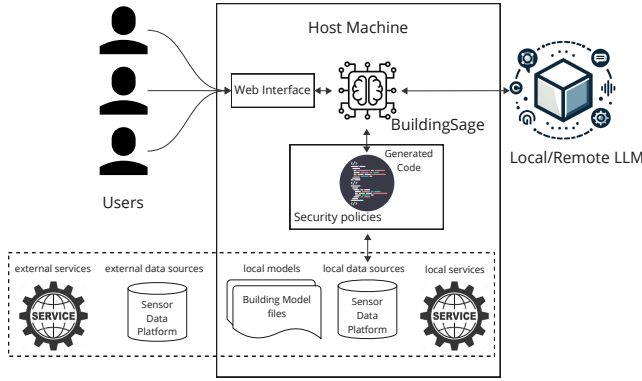


Figure 2: Architecture of the BuildingSage AI copilot.

The AI copilot system operates on a host machine and features a modular design that supports a variety of functions tailored for smart building management. At the forefront of the system is the *Web Interface*, which serves as the primary interaction point for users. This interface is intuitively designed to facilitate the input of natural language instructions, configuration of data sources, analytics viewing, and initiating code generation through the BuildingSage components.

Central to the system’s functionality is the *BuildingSage*, which acts as the central AI engine. The BuildingSage is responsible for interpreting user inputs and translating them into actionable tasks. It can engage with either local or cloud-based LLMs to generate executable code for data processing.

The system’s integrity and data confidentiality are safeguarded by robust *Security Policies* that review the code generated by the BuildingSage. These policies are critical for preventing unauthorized data manipulation and mitigating the risk of sensitive data leakage, as further detailed in Section 3.2.

Furthermore, the architecture supports extensive data integration capabilities, seamlessly incorporating local data from Building Modeling files and sensory inputs from edge devices, along with services both externally or locally.

3.2 System Properties

The copilot system exhibits key properties essential for its effective application in smart building contexts:

Multi-agent framework A series of recent works reveal that once the LLMs are instantiated into separate agents with different but complimenting goals and behaviours, they could demonstrate capabilities far beyond a single agent and can accomplish complex tasks collectively [16, 17]. Similar principles are hence adopted in AI-agent-based automation frameworks such as AutoGen [26]. The proposed copilot system operates as a multi-agent system which leverages the collective intelligence and specialized profiles and skills of different agents to process complicated tasks effectively. For example, a *Coder Agent* specializes in generating executable code, while a *Code Inspector Agent* works to increase the code success rate. A *Data Wizard Agent* aids in integrating data and services into the context of code generation. A *User Proxy Agent* executes the code in a containerized manner on the host machine.

And oftentimes, an *Orchestrator Agent* is in place to coordinate interactions among the agents. Different agents communicate with LLMs in different ways to handle specific tasks within the workflow. The behavior of these agents is guided by prompt engineering.

For example, to tailor the *Coder Agent* for code generation specific to smart building applications, it can be configured with system-level prompts such as:

- * You are proficient in the BRICK schema and capable of writing SPARQL queries for smart buildings.
- * Assess users' requests to determine whether they can be addressed directly or require the execution of code.
- * Provide Python code or shell scripts for users to execute, complete with adequate documentation and error handling.
- * Utilize Python with RDFlib to perform SPARQL queries directly in this environment.
- * Employ Plotly for visualizations, noting any performance considerations in cases of large-scale data.
- * Save all output files in the './files/' folder.

Similarly, the *Code Inspector Agent* is prompted with a more protective role:

- * You are a code inspector responsible for detecting potential problems in code snippets.
- * Analyze code to identify syntax errors, logical errors, and potential security vulnerabilities.
- * Issue a warning if operations involving deletion or unauthorized writing of user data is detected.
- * Provide suggestions for best practices and code improvements to enhance security and efficiency.
- * Encourage adherence to coding standards and conventions for better readability.

This configuration encourages the agent to vigilantly monitor for security and integrity breaches in the code it reviews, safeguarding against potential vulnerabilities or misuse of data.

These prompts, presented here in a simplified form due to page limitations, enable substantial guidance over the behaviour of each agent, ensuring that they fulfil their roles while collaborating towards the system’s overarching goals. While it is impossible to precisely control an agent’s behaviours, carefully devised prompts allow us to guide their behaviours with high accuracy.

Privacy preservation In the domain of smart building systems, maintaining sensitive data confidentiality is crucial. The potential for sensitive data leakage when using cloud-based LLMs for processing private data is a significant concern. To address this, our system employs a strict meta-data-only policy to shield local data from remote LLMs. Instead of transmitting raw data, only the schema or format of the data is shared, ensuring no sensitive content is disclosed.

For example, consider an application focused on occupant comfort, where it is necessary to query all rooms equipped with temperature and occupancy sensors to determine which room provides the best comfort level based on the room temperature and number of occupants. Instead of providing the LLMs with the sensitive and private raw data of temperature readings and occupant counts directly, the copilot system only sends the meta-data in the format of an abstract data schema that outlines the types of sensors and the nature of the measurements, such as `sensor_type: 'temperature'`, `measurement_type: 'celsius'` and `sensor_type: 'occupancy'`, `measurement_type: 'count'`. The LLMs then use this schema to generate a query statement using SPARQL, which is a query language used for extracting information from building models regarding the set of resources and their relationships that are required to operate building applications [6]:

```
SELECT ?room WHERE {
  ?room <hasSensor> ?tempSensor .
  ?tempSensor <sensorType> "temperature" .
  ?tempSensor <measurement> ?temperature .
  ?room <hasSensor> ?occupSensor .
  ?occupSensor <sensorType> "occupancy" .
  ?occupSensor <measurement> ?occupancy .
  FILTER (?temperature > 22 && ?temperature < 26
    && ?occupancy <= 5)
}
```

This query is designed to find rooms where the temperature is within a comfortable range and the occupancy is not too high. The user can execute this query on their local database, ensuring that no sensitive or private data is disclosed to the LLMs during the interaction.

Capabilities The Copilot system is equipped with a comprehensive array of capabilities designed to meet the diverse needs of smart building energy systems. It excels in integrating various data sources, including web services, local codes, Building Modeling files, and sensory data from building infrastructure sensors. By aggregating these data streams, the system provides a holistic view of building operations, enabling more informed decision-making and efficient energy management.

At the core of the copilot system's functionality is its ability to use programming code as a medium to synthesize information from disparate sources. This capability is pivotal in executing complex tasks that require data correlation and analysis across different services.

Additionally, the system is designed to be accessible to users with varying levels of technical expertise, including those with minimal programming knowledge. It processes natural language instructions to generate and execute necessary code automatically, significantly lowering the barrier to complex data science tasks and allowing users to concentrate on strategic decisions.

4 EXAMPLE USE CASE

This section presents an end-end use case scenario, demonstrating information retrieval, reasoning, natural language understanding, code generation and execution, and self-correction capabilities of

the proposed AI copilot in assisting users with smart building energy tasks in a privacy-preserving manner.

In the use case demonstrated in Figure 3, the user (i) generates SPARQL queries to explore data sources in a building and retrieve the stream ids of the determined data streams, runs the queries on the building model, (ii) generates the API scripts using the determined stream ids to retrieve the data streams from a data platform, (iii) generates visualizations using the data, and applies data analytics to detect anomalies with the assistance of the AI-copilot, using natural language.

Subsequent sections will introduce detailed use cases that highlight the diverse functionalities of BuildingSage. Throughout these demonstrations, we utilize the GPT-4 APIs as the LLM of choice to power the AI agents.

4.1 SPARQL query generation

Here we present a basic workflow where users, particularly those with limited expertise in coding and specific query languages such as SPARQL, interact with Brick building model files through the use of the AI copilot system. Brick is a schema used for the representation of building information based on a hierarchy of sensors, subsystems, and the relationships among them [3].

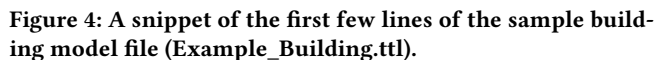
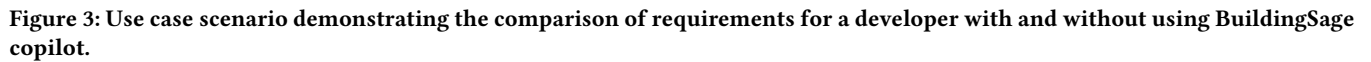
A snippet of the first few lines of the building model file, which encapsulates data for a building's heating systems and voltage sensors among other infrastructure components for the site designated as "Example_Site" within the organizational framework of "DCH-Demo", is illustrated in Figure 4. Utilizing the AI copilot system, a user can seamlessly incorporate this file as a data source within the system's interface.

The generation of SPARQL queries from natural language queries is a complex task that usually requires domain expertise. In [19], the authors propose an embedding technique to embed linguistic features of a question and graph information and generate SPARQL queries from natural language using GPT-2 without considering a specific domain.

Fig. 5 demonstrates how BuildingSage can assist SPARQL query generation relying on its capability to understand complex query requests in natural language and transform them into structured SPARQL queries for building models.

Upon initiating a query, the user submits a request to generate a SPARQL query aimed at identifying all floors within the "Example_Building" corresponding to the organizational ID "DCH-Demo" and Site ID "Example_Site". The User Proxy, acting on behalf of the user, forwards this request along with the metadata of the building model file to the Coder agent. The Coder, communicating with LLMs, formulates a pertinent code snippet based on two critical pieces of information: the user's request and the building model schema. Once generated, this query snippet is presented to the user for review and execution through the User Proxy.

Should the execution encounter errors, the User Proxy captures and relays the error log back to the Coder. The Coder then iteratively refines the code based on the provided error message. This error correction loop may persist through several iterations depending on the task's complexity. Ultimately, successful execution of the code allows the user to access the desired query output. The entire



Following the initial prompt, the user prompts for all the conference rooms in LevelW and the stream id of the air temperature sensor in one of these conference rooms. Figure 6 demonstrates the final outputs generated by BuildingSage.

4.2 Generating scripts to retrieve data from an API of a data platform

Once the stream id corresponding to the air temperature sensor is retrieved by BuildingSage, the user needs to query the cloud data platform through the API of the data platform using the stream id. Traditionally, this requires the user to have broad knowledge of APIs for the data platform and deep knowledge of the programming language for the APIs. BuildingSage assists users in generating the scripts to retrieve and download data streams from the data platform based on natural language queries.

In this demonstration, we use ANONYMIZED¹ as the cloud data platform, which is a digital platform for storing, managing, and extracting valuable insights from smart building data. To guide BuildingSage in generating scripts for API calls to a specific data platform, the API format is shared with BuildingSage in the description of the cloud data source:

The cloud data platform stores the sensor data streams. When requested to retrieve data from the data platform, you generate a CURL script.

Example:

```

For the API_KEY and STREAM_ID,
START_DATE="2022-06-05T00:00:07.078Z"
END_DATE="2022-06-05T04:00:07.078Z"
FILENAME="senaps_observations_${STREAM_ID}.json"

```

```
curl -X GET "https://ANONYMIZED/api/sensor/v2
/observations?streamid=${STREAM_ID}&start=
${START_DATE}&end=${END_DATE}" \
-H "accept: */*" \
-H "apikey: ${API_KEY}" | jq '.results' > "${FILENAME}"
```

¹ANONYMIZED link

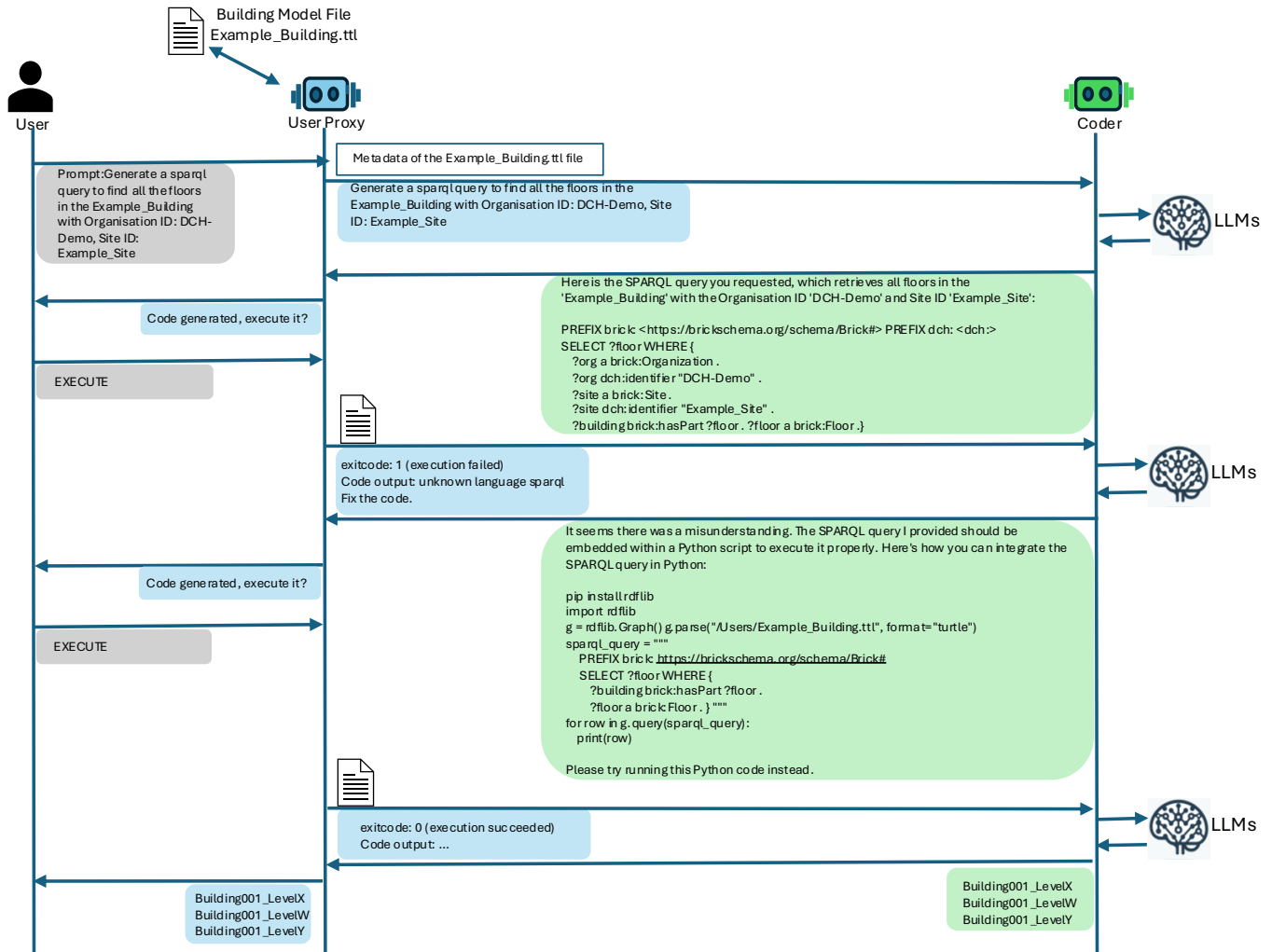


Figure 5: Workflow of using code generation to retrieve information from building model files.

To retrieve and download a sensor data stream, the user prompts BuildingSage to generate the script using the API template of the cloud data platform as demonstrated in Figure 7. Then, BuildingSage follows the template of the data cloud API and automatically formats the query accordingly. The generated CURL script is shared with the user. The user enters his API key and executes the CURL script, which downloads the data from the data platform.

4.3 Data processing and visualization

Visualizing and processing sensor data streams requires extensive knowledge of programming languages and data science, as well as significant effort in coding, debugging, and other related tasks. BuildingSage can assist users in data visualization and analytics tasks by generating the code required for visualizations, suggesting data processing methods, and generating the codes required for data processing. Then, the generated codes can be executed on local data files without compromising the privacy of the sensitive data. As described in Section 3, only the metadata (eg, the format) of the data file is shared with the remote LLM. For instance, for the downloaded air temperature sensor data, the format of the JSON file is shared in the description of the data source as:

BuildingSage: A safe and secure AI copilot for smart buildings

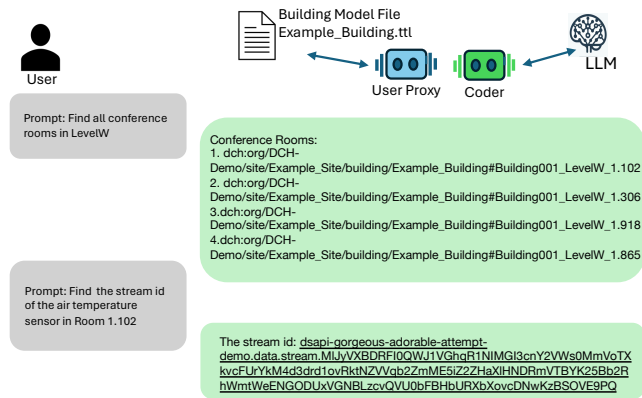


Figure 6: Retrieving the stream id of the air temperature sensor in a conference room.

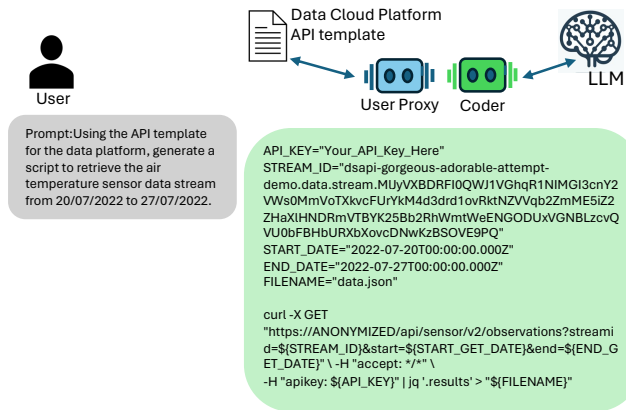


Figure 7: CURL script to retrieve sensor data stream from the data platform generated by BuildingSage.

The air temperature sensor data is stored in a JSON file.
The unit of temperature is Celsius.
The format of the file is:

```
[
  {
    "t": "TIMESTAMP",
    "v": {
      "v": VALUE
    }
  }
]
```

As demonstrated in Figure 8, BuildingSage generates data visualizations by generating code based on the description of the data file and executing the code locally on the data file. Furthermore, BuildingSage can assist users in data analytics tasks by suggesting methods, code generation and execution, and interpretation of the results. In Figure 9, BuildingSage assists the user with an anomaly detection task by suggesting appropriate anomaly detection methods, implementing the chosen method, and executing the code

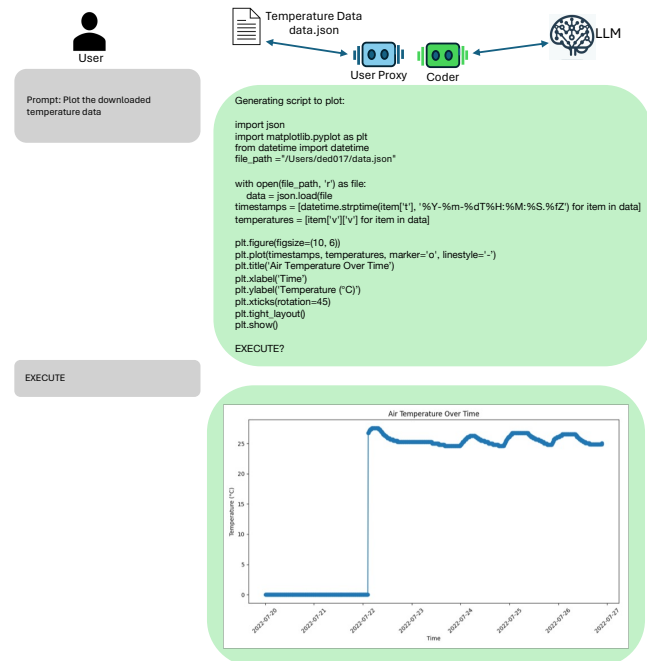


Figure 8: Data visualization with BuildingSage

on the temperature data. It generates and interprets the output, allowing the user to complete the task without any need for coding.

5 CHALLENGES AND LIMITATIONS

While generative AI offers numerous benefits for smart building energy systems, there are various challenges that need to be addressed for successful integration.

Security and privacy. Smart building energy systems can often involve sensitive data regarding occupant behaviours and usage patterns. Thus, secure and privacy-preserving data governance practices should be implemented while integrating generative AI. Running generative AI models locally, rather than relying on cloud-based solutions, enhances privacy and security and allows better control over data management. However, it requires significant hardware resources, and poses challenges in scalability and maintenance, necessitating frequent updates and requiring on-site technical expertise. Security and privacy can also be improved by leveraging generative AI to create code for specific tasks, which is then executed locally on data within a secure environment.

Computational challenges. Training or fine tuning generative AI models for building energy systems requires significant resources and domain-specific data.

Scalability. Building energy systems produce vast amounts of data, including temperature, humidity, and occupancy levels, making efficient data management a significant challenge. Furthermore, deploying generative AI models across diverse buildings raises scalability issues, due to the differences in design, usage, and environmental conditions. To address the scalability issues, more complex architectures and diverse training data are required, increasing the computational demands and complexity of implementation.

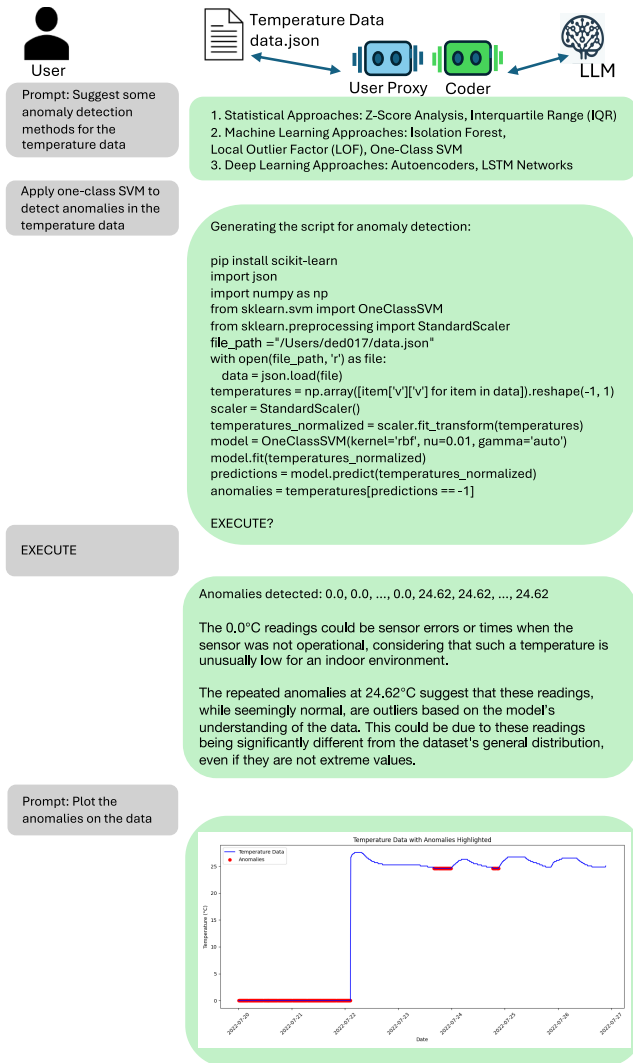


Figure 9: Anomaly detection with BuildingSage.

Latency. Smart building energy applications such as HVAC control require real-time data processing. Running large models on the cloud with such low latency is technically demanding. Locally deployed models reduce the latency and decrease the dependency on external communications.

Integration. Integrating generative AI into diverse smart building energy systems is challenging due to varying communication standards and protocols across different devices. Furthermore, the integration of generative AI into smart building energy systems is further complicated by the need to adhere to regulatory compliance standards.

Accuracy. There is often a trade-off between the accuracy and computational efficiency of a generative model. Furthermore, several key factors affect the accuracy, including the complexity of the models, the availability of computational resources, the quality of

training data, the effectiveness of prompting techniques, and the integration of human-in-the-loop optimization strategies.

Energy consumption. Large models are inherently energy-intensive, which poses a dilemma for energy systems that prioritize sustainability. When considering training and deployment of smaller models, there is a significant trade-off between the energy used for model training versus prediction. Training typically consumes more energy than prediction, and if a model is infrequently used for prediction, the energy expended during training could become the predominant cost factor.

6 CONCLUSION

In this paper, we have introduced BuildingSage, an innovative AI copilot designed to enhance the interaction between users and smart building systems, while addressing critical concerns about data privacy and security. BuildingSage addresses key challenges in building data analysis by enabling seamless interactions with smart building systems through natural language. Our implementation demonstrates that BuildingSage effectively bridges the gap between complex smart building technologies and users, leveraging the power of generative AI to streamline operations, enhance user experience, and maintain rigorous data privacy standards.

Through various use cases, we have illustrated how BuildingSage assists users in effortlessly integrating and interacting with smart building systems. The system's capability to generate SPARQL queries, automate integration with local and external sources and services, and facilitate data visualization and analysis, significantly reducing the technical barriers traditionally associated with smart building management. By democratizing access to complex system interactions, BuildingSage empowers users to optimize energy efficiency, enhance building operations, and improve occupant comfort without requiring deep technical expertise.

The continued evolution of generative AI promises further advancements in smart building technologies. We envision future developments that could include more intuitive user interfaces, enhanced predictive capabilities on multi-modalities such as imagery data, more computationally-efficient LLM agents, and even greater levels of customization and scalability. These advancements are likely to drive broader adoption and more innovative applications of AI in the smart building sector.

However, the deployment of such systems is not without challenges. Our future research will focus on enhancing the security features of AI copilots such as maintaining consistency in the outputs generated by generative AI, improving the efficiency of data handling and processing, expanding the adaptability of these systems to diverse architectural and operational contexts, and providing reliable benchmarks for performance evaluation of such AI systems.

BuildingSage is a step toward realizing the potential of AI in smart building management. By continuing to refine and advance this technology, we can unlock even more value, creating smarter, more responsive environments that adapt to and anticipate the needs of their occupants.

REFERENCES

- [1] Ki Uhn Ahn, Deuk-Woo Kim, Hyun Mi Cho, and Chang-U Chae. 2023. Alternative Approaches to HVAC Control of Chat Generative Pre-Trained Transformer

- (ChatGPT) for Autonomous Building System Operations. *Buildings* 13, 11 (2023), 2680.
- [2] Mahathir Almashor, Yusuke Miyashita, Sam West, and Thi Van Dai Dong. 2024. Can Private LLM Agents Synthesize Household Energy Consumption Data?. In *Proceedings of the 15th ACM International Conference on Future and Sustainable Energy Systems*. 664–668.
 - [3] Bharathan Balaji, Arka Bhattacharya, Gabriel Fierro, Jingkun Gao, Joshua Gluck, Dezhi Hong, Aslak Johansen, Jason Koh, Joern Ploennigs, Yuvraj Agarwal, Mario Berges, David Culler, Rajesh Gupta, Mikkel Baun Kjærsgaard, Mani Srivastava, and Kamin Whitehouse. 2016. Brick: Towards a Unified Metadata Schema For Buildings. In *Proceedings of the 3rd ACM International Conference on Systems for Energy-Efficient Built Environments* (Palo Alto, CA, USA) (*BuildSys '16*). Association for Computing Machinery, New York, NY, USA, 41–50. <https://doi.org/10.1145/2993422.2993577>
 - [4] Marco Cascella, Jonathan Montomoli, Valentina Bellini, and Elena Bignami. 2023. Evaluating the feasibility of ChatGPT in healthcare: an analysis of multiple clinical and research scenarios. *Journal of Medical Systems* 47, 1 (2023), 33.
 - [5] Laura Daniele, Frank den Hartog, and Jasper Roes. 2015. Created in close interaction with the industry: the smart appliances reference (SAREF) ontology. In *Formal Ontologies Meet Industry: 7th International Workshop, FOMI 2015, Berlin, Germany, August 5, 2015, Proceedings 7*. Springer, 100–112.
 - [6] Gabe Fierro and David E. Culler. 2018. Design and Analysis of a Query Processor for Brick. *ACM Trans. Sen. Netw.* 14, 3–4, Article 18 (nov 2018), 25 pages. <https://doi.org/10.1145/3199666>
 - [7] Gabe Fierro, Jason Koh, Yuvraj Agarwal, Rajesh K. Gupta, and David E. Culler. 2019. Beyond a House of Sticks: Formalizing Metadata Tags with Brick. In *Proceedings of the 6th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation* (New York, NY, USA) (*BuildSys '19*). Association for Computing Machinery, New York, NY, USA, 125–134. <https://doi.org/10.1145/3360322.3360862>
 - [8] Gihan Gamage, Sachin Kahawala, Nishan Mills, Daswin De Silva, Milos Manic, Daminda Alahakoon, and Andrew Jennings. 2023. Augmenting industrial chatbots in energy systems using chatgpt generative ai. In *2023 IEEE 32nd International Symposium on Industrial Electronics (ISIE)*. IEEE, 1–6.
 - [9] Karl Hammar, Erik Oskar Wallin, Per Karlberg, and David Hälleberg. 2019. The realstatecore ontology. In *The Semantic Web—ISWC 2019: 18th International Semantic Web Conference, Auckland, New Zealand, October 26–30, 2019, Proceedings, Part II 18*. Springer, 130–145.
 - [10] Yushan Jiang, Zijie Pan, Xikun Zhang, Sahil Garg, Anderson Schneider, Yuriy Nevmyvaka, and Dongjin Song. 2024. Empowering Time Series Analysis with Large Language Models: A Survey. *arXiv:2402.03182* [cs.LG]
 - [11] J John et al. 2020. Project haystack data standards. In *Energy and Analytics*. River Publishers, 237–243.
 - [12] Enkelejda Kasneci, Kathrin Seifler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günemann, Eyke Hüllermeier, et al. 2023. ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and individual differences* 103 (2023), 102274.
 - [13] Vimal Kumar, Priyam Srivastava, Ashay Dwivedi, Ishan Budhiraja, Debjani Ghosh, Vikas Goyal, and Ruchika Arora. 2023. Large-Language-Models (LLM)-Based AI Chatbots: Architecture, In-Depth Analysis and Their Performance Evaluation. In *International Conference on Recent Trends in Image Processing and Pattern Recognition*. Springer, 237–249.
 - [14] Na Luo, Marco Pritoni, and Tianzhen Hong. 2021. An overview of data tools for representing and managing building information and performance data. *Renewable and Sustainable Energy Reviews* 147 (2021), 111224. <https://doi.org/10.1016/j.rser.2021.111224>
 - [15] Filippos Lygerakis, Nikos Kampelis, and Dionysia Kolokotsa. 2022. Knowledge Graphs' Ontologies and Applications for Energy Efficiency in Buildings: A Review. *Energies* 15, 20 (2022). <https://doi.org/10.3390/en15207520>
 - [16] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (<conf-loc>, <city>San Francisco</city>, <state>CA</state>, <country>USA</country>, </conf-loc>) (*UIST '23*). Association for Computing Machinery, New York, NY, USA, Article 2, 22 pages. <https://doi.org/10.1145/3586183.3606763>
 - [17] Chen Qian, Xin Cong, Wei Liu, Cheng Yang, Weize Chen, Yusheng Su, Yufan Dang, Jiahao Li, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. Communicative Agents for Software Development. *arXiv:2307.07924* [cs.SE]
 - [18] Caroline Quinn and J.J. McArthur. 2021. A case study comparing the completeness and expressiveness of two industry recognized ontologies. *Advanced Engineering Informatics* 47 (2021), 101233. <https://doi.org/10.1016/j.aei.2020.101233>
 - [19] Md Rashad Al Hasan Rony, Uttam Kumar, Roman Teucher, Liubov Kovriguina, and Jens Lehmann. 2022. SGPT: A Generative Approach for SPARQL Query Generation From Natural Language Questions. *IEEE Access* 10 (2022), 70712–70723. <https://doi.org/10.1109/ACCESS.2022.3188714>
 - [20] Fardin Ahsan Sakib, Saadat Hasan Khan, and AHM Karim. 2023. Extending the frontier of chatgpt: Code generation and debugging. *arXiv preprint arXiv:2307.08260* (2023).
 - [21] Ankita Sharma, Xuanmao Li, Hong Guan, Guoxin Sun, Liang Zhang, Lanjun Wang, Kesheng Wu, Lei Cao, Erkang Zhu, Alexander Sim, Teresa Wu, and Jia Zou. 2023. Automatic Data Transformation Using Large Language Model - An Experimental Study on Building Energy Data. In *2023 IEEE International Conference on Big Data (BigData)*. 1824–1834. <https://doi.org/10.1109/BigData59044.2023.10386931>
 - [22] Lei Song, Chuheng Zhang, Li Zhao, and Jiang Bian. 2023. Pre-trained large language models for industrial control. *arXiv preprint arXiv:2308.03028* (2023).
 - [23] Jing Su, Chufeng Jiang, Xin Jin, Yuxin Qiao, Tingsong Xiao, Hongda Ma, Rong Wei, Zhi Jing, Jiajun Xu, and Junhong Lin. 2024. Large Language Models for Forecasting and Anomaly Detection: A Systematic Literature Review. *arXiv:2402.10350* [cs.LG]
 - [24] Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. 2023. On the Planning Abilities of Large Language Models - A Critical Investigation. In *Advances in Neural Information Processing Systems*, A. Oh, T. Neumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 75993–76005. https://proceedings.neurips.cc/paper_files/paper/2023/file/efb2072a358cefb75886a315a6fcf880-Paper-Conference.pdf
 - [25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ec243547dee91fbd053c1c4a845aa-Paper.pdf
 - [26] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155* (2023).
 - [27] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhajan Kambadur, David Rosenberg, and Gideon Mann. 2023. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564* (2023).
 - [28] Zhipeng Xu, Hao Wu, Xu Chen, Yongmei Wang, and Zhenyu Yue. 2022. Building a Natural Language Query and Control Interface for IoT Platforms. *IEEE Access* 10 (2022), 68655–68668. <https://doi.org/10.1109/ACCESS.2022.3186760>
 - [29] Hao Xue and Flora D Salim. 2023. Utilizing Language Models for Energy Load Forecasting. In *Proceedings of the 10th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*. 224–227.
 - [30] Chaobo Zhang, Jie Lu, and Yang Zhao. 2024. Generative pre-trained transformers (GPT)-based automated data mining for building energy management: Advantages, limitations and the future. *Energy and Built Environment* 5, 1 (2024), 143–169. <https://doi.org/10.1016/j.enbenv.2023.06.005>
 - [31] Liang Zhang and Zhelun Chen. 2023. Opportunities and Challenges of Applying Large Language Models in Building Energy Efficiency and Decarbonization Studies: An Exploratory Overview. *arXiv preprint arXiv:2312.11701* (2023).

Received 23 July 2024